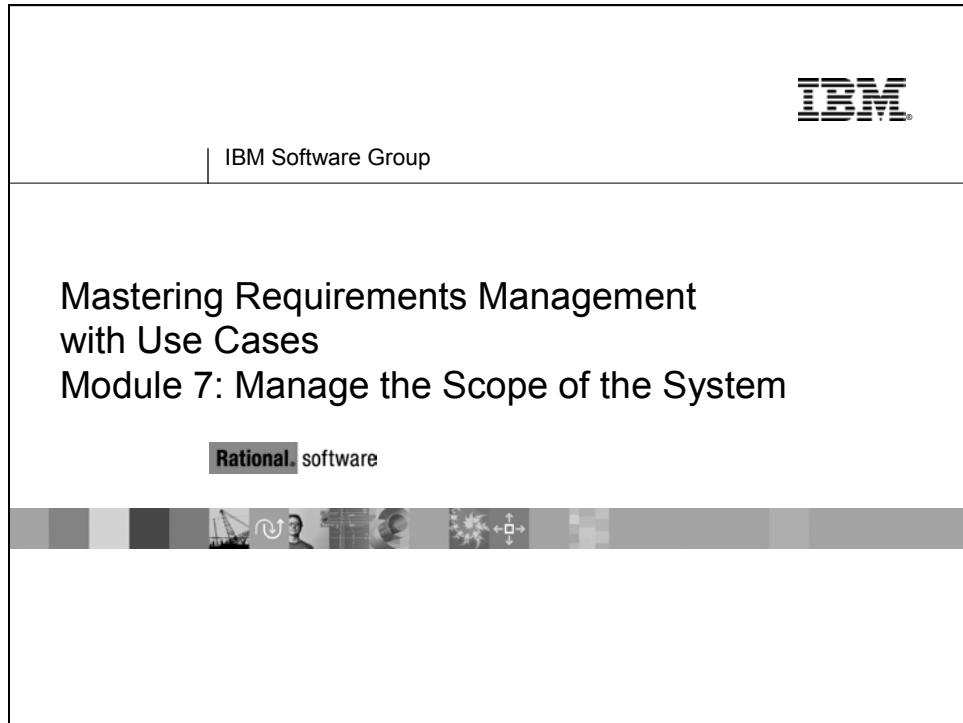


► ► ► **Module 7**

Manage the Scope of the System



Topics

Scope Management.....	7-5
Define the System Scope.....	7-6
Establish Requirements Baseline	7-7
Use Cases are Written and Implemented Iteratively	7-9
Prioritize the Use Cases	7-10
Process Helps Manage Scope	7-13
Manage Expectations.....	7-14
Improve Your Negotiation Skills.....	7-16
The Product Champion	7-17

Objectives

Objectives

- ♦ Describe the steps to manage scope.
 - Define system scope (resources, budget, time).
 - Establish requirements baseline.
 - Use attribute values to prioritize requirements.
 - Prioritize use cases.
- ♦ List ways to manage expectations.

2

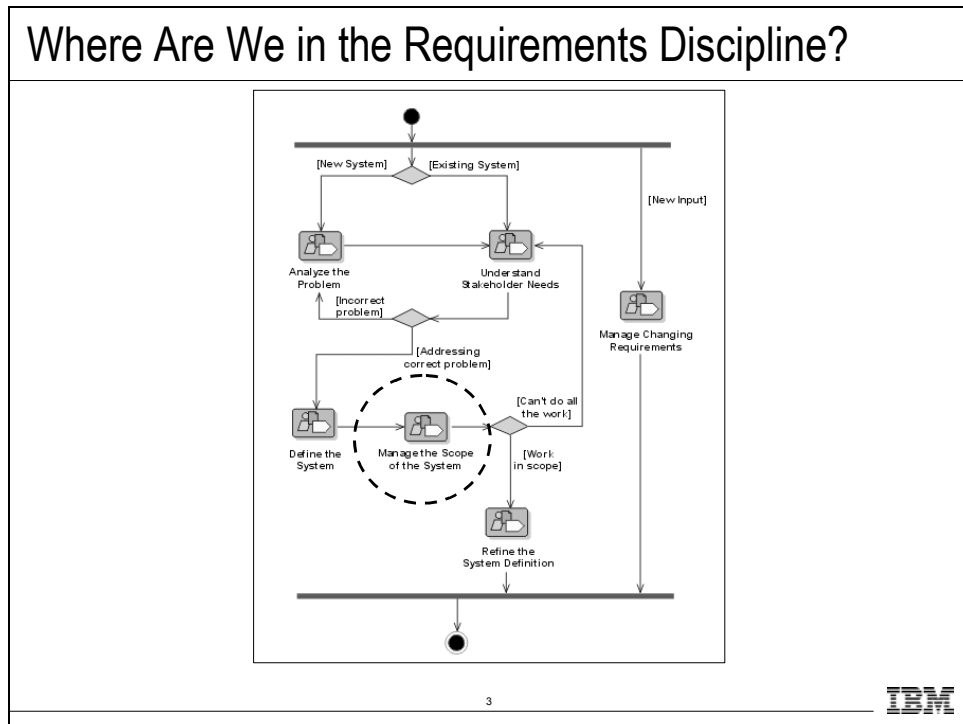


To some, scope management means change management. In the RUP® it is used in the sense of setting boundaries for each iteration.

Scope management is maintaining the “healthy tension” between what the customer wants (maximum features) and what developers believe they can deliver in a fixed time frame. It is essential for the developer to get the agreement of the customer on the baseline set of features to develop in the system to be built.

A good way to achieve the balance between customer desires and developer resources is by using iterative development and providing incremental “slices of the pie.” That way, the priorities can be reassessed at the end of each iteration.

Where Are We in the Requirements Discipline?



In this module, we are focusing on prioritizing and setting a baseline scope for the system development in the current release.

The ideal time to decide on what is in the baseline scope of features we actually develop is after the system is defined (Module 6) and before we have put a lot of time into refining the details (Module 8).

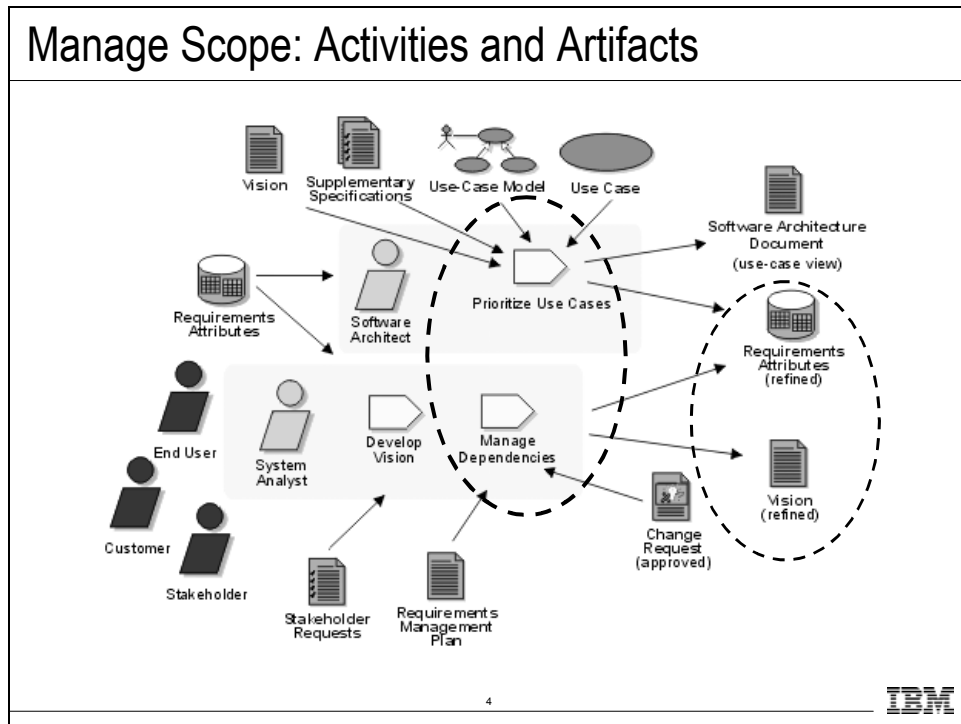
Where does Manage the Scope of the System workflow detail fit into our development process?

The Managing Scope Workflow Detail in the RUP® is performed after the system is defined, but before refining the definition. We do not want to waste time refining parts of the system that are outside the scope of the current version of the system.

Where is this done in your own software development process?

Remember, the diagram above shows just one iteration of the process, not the entire requirements management lifecycle.

Manage Scope: Activities and Artifacts



In RUP ®, the purpose of this workflow detail is to:

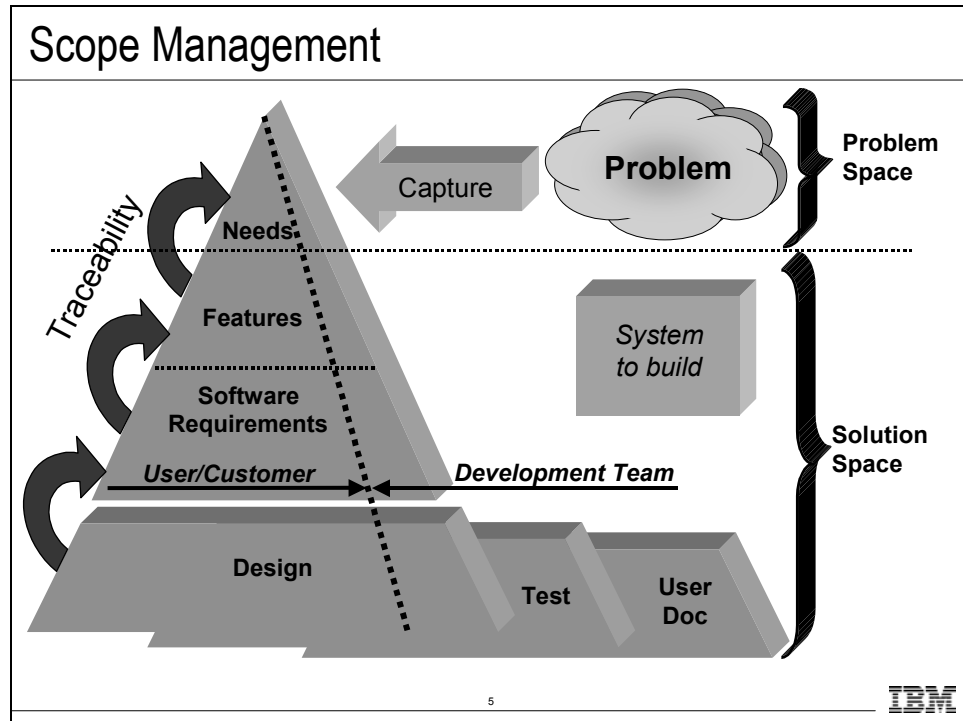
- Define input to the selection of requirements that are to be included in the current iteration.
- Define the set of features and use cases (or scenarios) that represent some significant, central functionality.
- Define which attributes and traceabilities to maintain.

Although project scope should be managed continuously, it is easier to make informed decisions after identifying most actors, use cases, and supplementary specifications. The system analyst applies customer priority, effort, cost, risk values, and other requirements attributes to more accurately rank the development priorities. This enables the architect to identify the architecturally significant use cases.

The Iteration Plan is developed in parallel by the Project manager. The Iteration Plan defines the number and frequency of iterations planned for the release.

The scope of the project defined in Managing Scope has a significant impact on the Iteration Plan because the highest risk elements within scope are planned for early iterations. Other important outputs from Managing Scope include the initial iteration of the Software Architecture Document and a revised Vision that reflects system analysts' and key stakeholders' better understanding of system functionality and project resources.

Scope Management



Scope management is maintaining a “healthy tension” between what the customer wants (maximum features) and what development believes it can deliver in a fixed timeframe. It is essential for the developer to get agreement from the customer regarding a baseline set of features to develop for the system to be built.

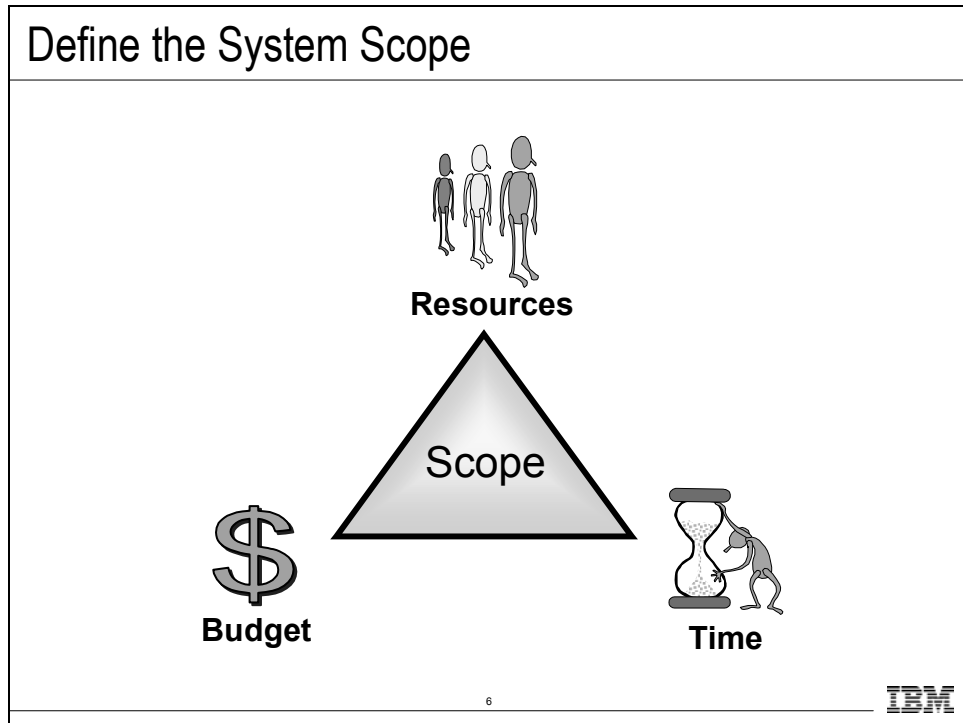
A good way to achieve the balance between customer desires and developer resources is by using iterative development and providing incremental “slices of the pie.” Then, the priorities can be reassessed at the end of each iteration.

The ideal time to decide what the baseline set of features we will actually develop is after the system is defined (Module 5) and before we have put a lot of time into refining the details (Module 7).

The following is a list of tips for avoiding scope creep, but allowing change:

- ‘Real’ requirements: Identify what is really needed for the business objective-prototyping, iterative development.
- Minimum requirements: A conscious practice of a minimum requirements strategy, no ‘gold plating’, no including what ‘might be needed’.
- All requirements should be recorded and identified by source.
- Realize all requirements have a cost and schedule impact.
- The use of an agreed negotiation and sanctioning process (it need not be heavyweight).
- All requirements come (in the end) through a well-identified channel, not from various and sundry random sources.
- Expectation management and communication with the customer about what will be in each iteration. This helps uncover hidden requirements, unknown to the naïve developer but assumed by the domain-experienced customer.

Define the System Scope



In this module, you focus on what can be done realistically to take the fixed amount of resources and choose the best value to produce for the customers. Of course there's always overtime, but you also want to avoid eventual burnout and be in this for the long haul, pacing yourselves for a marathon, rather than a sprint.

The scope of a project is defined by the set of requirements allocated to it. Managing project scope to fit the available resources (time, people, and money) is key to managing successful projects. Managing scope is a continuous activity that requires iterative or incremental development, which breaks project scope into smaller, more manageable pieces.

Establish Requirements Baseline

Establish Requirements Baseline

Requirements Baseline
 The set of features that constitutes the agreed upon basis for development and that can only be changed through a formal procedure.



How do we know what the needs are?
 How do we determine priority?
 Where do we set the baseline?

- ◆ Feature 1: The system must...
- ◆ Feature 2: The system must...
- ◆ Feature n: The system must...



Project Start Date Target Release Date Time

IBM

How do we reach agreement on what features should be included in our project?
 Where would we set our baseline commitment for delivery?

The key is to under-promise and over-deliver but not too much! We want to maintain our credibility.

What needs of our stakeholders do the features represent?

How should features be prioritized?

What factors might influence the order in which we rank the features?

Uses for Requirement Attributes

Uses for Requirement Attributes					
Attributes link project elements					
	Status	Risk	Priority	Effort	Cost
FEAT 10	Approved 	Low	High 		\$\$\$
FEAT 13	Proposed	Medium	Low	 	\$\$
FEAT 40	Approved 	High 	High 		\$

 = Filter

IBM

Once you have a list of features in the baseline scope, the features need to be allocated to iterations so they can be implemented in a manner that removes the project risk and delivers important functionality as early as possible. Attributes play a key role in this decision process.

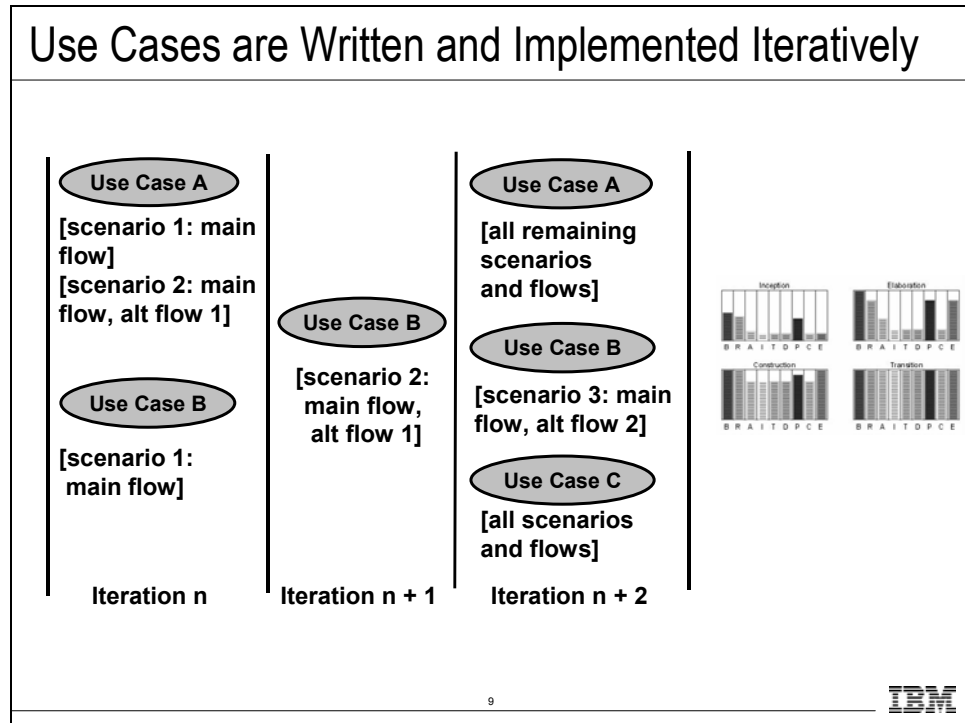
Requirement attributes provide a link between requirements and other project elements. For example, “status” provides the current status of each requirement and “effort” estimates the work involved to do each requirement. Among the uses for requirement attributes are:

- Managing project scope
- Assigning resources
- Scheduling
- Assessing status
- Calculating software metrics
- Managing project risk
- Estimating costs
- Assuring user safety

Attributes can be specified for all types of requirements: needs, features, use cases, supplementary requirements. The attributes to collect at each level are based on what information is needed for management reports and other uses.

The funnel icons represent a typical filter that an architect may apply when analyzing requirements to help make management decisions. In the above example the filter would be: “Show me all requirements with a status of APPROVED and a risk of HIGH RISK and a priority of HIGH and an effort of LOW.”

Use Cases are Written and Implemented Iteratively



Detailing random flows in a use case results in a fragmented specification that cannot really be implemented and tested successfully. When you detail flows in an iteration, it makes sense to detail them so that each flow forms part of a complete story (scenario) that can be implemented and tested. Therefore, based on selected scenarios for a given iteration, you elaborate the flows that are part of specific scenarios. Developers and testers can then use these scenarios to implement and test.

A given use case is usually not completely written and implemented in a single iteration. Instead, each iteration focuses on a subset of the use-case scenarios. The scope of the iteration is driven by several factors:

- The top risks to the project.
- The functionality required of the system.
- The time allocated to the iteration.
- The phase and its specific objectives.

The more architecturally significant scenarios are addressed in the early iterations.

Prioritize the Use Cases

Prioritize the Use Cases



Architect

1. Consider use cases linked to features in the baseline scope.
2. Select the scenarios for architectural iterations based on flows that:
 - Represent significant, central functionality.
 - Have a substantial architectural coverage.
 - Exercise many architectural elements/interfaces.
 - Stress a specific, delicate point of the architecture.
 - Have been identified as high risk.
3. Prioritize scenarios for future iterations.



RUCS9:
 Software
 Development
 Plan



10

The first few iterations in a project are sometimes called the architectural iterations because these iterations are used to test out the architecture.

Based on the use cases tied to features in the baseline scope, the software architect decides how to order or prioritize the use cases or scenarios across the iterations. It is very important to establish a sound architecture in the early iterations, so that it provides a solid foundation for the rest of the project. Therefore, use-case scenarios of architectural significance are done first. How does the architect decide what use case(s) are of architectural significance? The list on the slide is the main criteria that the architect will use in making these decisions.

Architecturally significant scenarios are discovered by looking at the flows that describe the most architecturally significant functionality. The architect then maps those flows to one or more scenarios. Those use-case scenarios are assigned to the first architectural iteration.

After deciding on use cases scenarios for the first iteration, you decide the approximate sequence for future iterations using the same criteria described above. At the beginning of the project, tentative priorities are assigned to all the use cases. But, during the assessment at the end of every iteration, you may need to revise the order in which use cases are assigned to future iterations.

As the project moves from phase to phase, the criteria for prioritizing use cases changes. During Elaboration you select use cases or scenarios to mitigate the technical as quickly as possible. During Construction you will select use cases or scenarios to deliver essential functionality before non-essential functionality (80:20 rule).

Prioritize the Use Cases (cont.)

Prioritize the Use Cases (cont.)	
 Analyst	1. Consider the priority of stakeholder requests and features in the baseline scope. 2. Select the scenarios for the iteration based on the flows that: ▪ Trace to high-priority stakeholder requests. ▪ Represent the main use of the system (80:20 rule). ▪ Are related to features that, once delivered, allow you to receive an incremental payment. ▪ Provide a key differentiator to major competitors. 3. Prioritize scenarios for future iterations.
	

The system analyst is interested in prioritizing use cases (or scenarios) based on high priority features that trace to different flows. There are many factors that influence how a system analyst prioritizes his/her use cases. Typically they are concerned with getting that 20 percent of functionality that will solve 80 percent of the stakeholder's needs implemented as soon as possible. By definition, these 20 percent are usually the highest priority requirements.

Systems analysts are not concerned with the technicalities of implementing a use case—that is the concern of the architect. For this reason, a balance between the system analyst's and the architect's desires must be reached.

Exercise 7.1: Prioritize Requirements Using Attributes

Exercise 7.1: Prioritize Requirements Using Attributes

- ♦ Review feature matrix and attributes.
- ♦ Determine relative importance of attributes.
- ♦ Prioritize features.
- ♦ Decide on baseline scope.



12

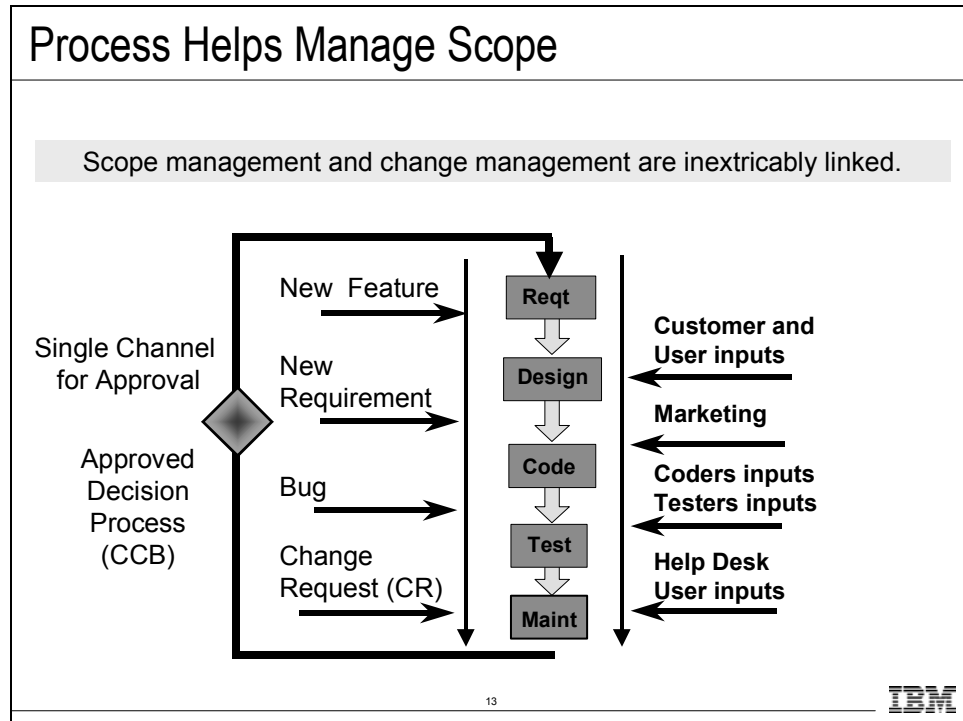
IBM

See Student Workbook Exercise 7.1: Use Attributes to Prioritize Requirements.

In this exercise you will use attributes to prioritize the features and determine the baseline scope of the system.

For ease of comparison, list all the requirements (not just 2/3); and list them in rank order (which to do first, second, etc.) rather than requirement order as on the matrix.

Process Helps Manage Scope



A key factor in managing scope is an effective change management process. As requests come in during our lifecycle, intercept them, and pass through a single approval channel.

Requests can then be assessed according to criteria such as: origin, customer priority, support of business goals, and impact on schedule.

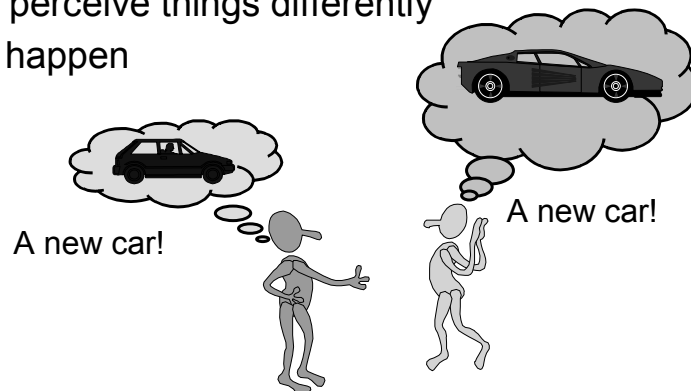
The single channel of approval may be one person (like a project manager) or perhaps a group of representatives (for example; a CCB, Change or Configuration Control Board). The CCB should have representatives from each of the relevant groups.

Manage Expectations

Manage Expectations

- ♦ Why manage expectations?

- So customers understand why you are deferring functionality
- People perceive things differently
- Things happen



Gause & Weinberg, 1989

14

IBM

One of the keys to having happy customers at delivery time is to manage their expectations of what they are to receive.

How to Manage Expectations

How to Manage Expectations

- ♦ Understand customer expectations.
- ♦ Limit the expectations as appropriate.
- ♦ Include the source of the limitation.
- ♦ Under-promise and over-deliver.

Keep the possibilities open.

15



Communication is very important; no surprises.

Improve Your Negotiation Skills

Improve Your Negotiation Skills

- ♦ Start negotiations with high demands, but don't be unreasonable.
- ♦ Separate the people from the problem.
- ♦ Focus on interests, not positions.
- ♦ Understand your Best Alternative To a Negotiated Arrangement (BATNA).
- ♦ Invent options for mutual gain.
- ♦ Use diplomacy.

Improve your skills at every opportunity.

Fisher, Ury, *Getting to Yes*, 1991

16



Negotiation skills are key to any successful, multi-party program. Improving these skills is a normal, professional activity that everyone should spend time on.

This information is from *Getting to Yes*, by Fisher and Ury. Fisher and Ury worked with the Harvard Negotiation Project, who studied high-level (mostly diplomatic) negotiations and evaluated what made them successful.

The concept of BATNA (Best Alternative To a Negotiated Agreement) encourages you to also look at the consequences of *not* getting an agreement. How important is that to you? What if the customer cancels the project? This gives you a bottom line from which to work.

The key is to focus on the interests of all involved and attempt to come up with creative options that would satisfy both sides.

The Product Champion

The Product Champion

- ◆ Prevents projects from drifting into a technical or political abyss.
- ◆ Helps manage project scope.
- ◆ Owns the product vision.
- ◆ Advocates for the product.
- ◆ Defends against feature creep.
- ◆ Negotiates with management, users, and developers.
- ◆ Maintains a balance between customer needs and what can be delivered on time.
- ◆ Represents the official channel between the customer and the development team.



17

IBM

The product champion is a very important, yet often intangible, aspect of having a successful project.

Usually this is not a job title, but a role played by a key individual.

Would you want to have a product champion on the development side or the customer side?

At what level in your company would this person need to be at to be effective?

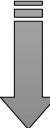
The notion of a product champion is supported by the research performed by the Standish Group (refer to Module 2). Their findings were: "Without a staunch project champion with a solid business vision, projects can drift into a technical or political abyss."

Exercise 7.2: Prioritize Scenarios

Exercise 7.2: Prioritize Scenarios



Analyst



Architect

- ✓ Look at the features in your baseline scope.
- ✓ Look at the use cases in your project.
- ✓ What use cases/scenarios would you choose in the first iteration? Why?
- ✓ What else should you consider?
- ✓ Scope your project.

- ✓ Look at the scope of the project.
- ✓ What use cases/scenarios should be implemented in the first iteration? Why?
- ✓ In what sequence should the use cases/scenarios be implemented?
- ✓ What else should you consider?



18



See Student Workbook Exercise 7.2: Prioritize Scenarios.

Review: Manage the Scope of the System

Review: Manage the Scope of the System

1. How does a CM process support scope management?
2. How are attributes useful in managing scope?
3. How much of a use case do you detail in an iteration?
4. What are the roles of the architect and the analyst in managing scope?
5. Why is it important to manage expectations? How is it done?
6. What is the role of the product champion?

19



Answers

1.

2.

3.

4.

5.

6.

